

API Authentication

 systemdesignschool.io/fundamentals/api-design-authentication



Modern applications require secure API authentication to protect resources and identify users. The choice between API keys, sessions, and JWTs depends on your scalability, security, and user context requirements.

API Keys: Application Identity

API keys authenticate applications, not users. Each client receives a unique key that identifies the calling service. This works well when you need to track which application is making requests, but don't need user-specific permissions.

How they work

Each API key acts like a password for your application. You include it in request headers to identify which application is calling the API. This enables rate limiting and usage tracking per application. GitHub gives higher rate limits to requests with valid tokens (5,000/hour) compared to anonymous requests (60/hour).

```
GET /v1/charges
```

```
Authorization: Bearer sk_test_4eC39HqLyjWDarjtT1zdp7dc
```

API Key Authentication




API Key stored securely
in client application

Simple but requires secure key management



Long-lived credentials

API keys typically last months or years, making lifecycle management critical. Production systems need key rotation strategies, usage monitoring, and revocation mechanisms. AWS provides tools to help teams rotate access keys and identify unused keys.

Sessions: Server-Side State

Sessions maintain server-side state for user interactions. When users authenticate, the server creates a session record containing user information and permissions. The client receives only a session identifier, typically as an HTTP cookie.

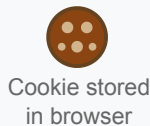
Rich user context

Sessions store user identifiers and lightweight state (user ID, role IDs, CSRF tokens). Keep sessions small and reference larger data from profile stores. This reduces memory pressure and simplifies invalidation. Banking applications use 15-minute timeouts, secure cookie flags (HttpOnly, Secure, SameSite), and immediate invalidation on suspicious activity.

Scaling challenges

Reddit migrated from in-memory sessions to Redis-backed sessions to support multiple application servers. This enabled horizontal scaling but introduced Redis as a critical dependency. Sessions work well for traditional web applications but complicate mobile and API clients.

Session Cookie Authentication



Server-side session management • Automatic cookie handling
Vulnerable to CSRF attacks • Requires secure transmission



Session Management with Security

JWT: Self-Contained Tokens

JWTs (JSON Web Tokens) contain encoded user information and permissions directly within the token itself. Unlike sessions, the server validates tokens cryptographically without database lookups, enabling stateless authentication that scales horizontally across distributed systems.

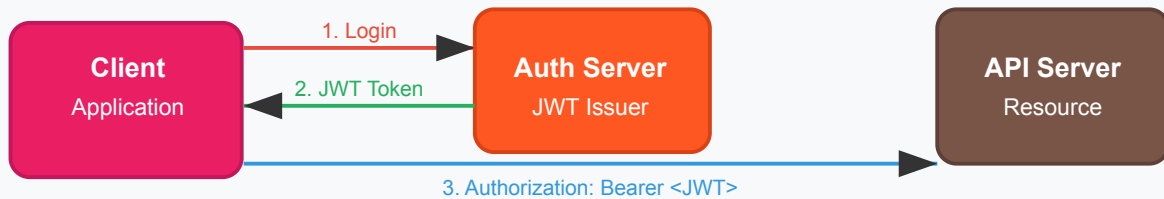
Claims and permissions

JWT tokens embed user claims like roles, permissions, and expiration times. When a client makes an API request, the server validates the token locally without database queries. However, services must still fetch public keys (JWKS) for signature verification and validate issuer, audience, and expiration with clock skew tolerance. Prefer role IDs over large permission sets to avoid token bloat.

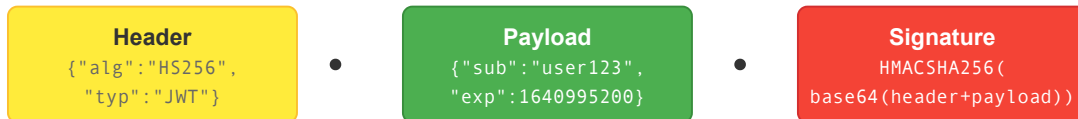
The revocation problem

Since tokens are self-contained, you can't immediately invalidate them like sessions. Production systems use short expiration times with refresh tokens, or maintain token blacklists for critical scenarios. Choose TTL based on risk: 5-15 minutes for high-risk applications, 15-60 minutes for typical web apps, hours for some mobile use cases.

JWT Bearer Token Authentication



JWT Structure



localStorage Token stored client-side

Stateless • Self-contained • Verifiable signature • Short expiration recommended



JWT Creation and Validation

```
from typing import Dict, Optional
def __init__(self, secret_key: str):
    self.secret_key = secret_key.encode()
def create_token(self, user_id: str, claims: Dict, expires_in: int = 3600) -> str:
    """Create JWT with user claims"""
    "alg": "HS256",
    # Payload with claims
    current_time = int(time.time())
    "user_id": user_id,
    "iat": current_time, # Issued at
    "exp": current_time + expires_in, # Expires
    **claims # Additional claims (roles, permissions, etc.)
    # Create token parts (use canonical JSON)
    header_b64 = base64.urlsafe_b64encode(
        json.dumps(header, separators=(",", ":")).encode()
    ).decode().rstrip('=')
    payload_b64 = base64.urlsafe_b64encode(
        json.dumps(payload, separators=(",", ":")).encode()
    ).decode().rstrip('=')
```

Choosing Your Authentication Strategy

Each pattern suits different use cases. API keys work for service-to-service authentication and rate limiting. Sessions fit traditional web applications needing rich user context. JWTs enable distributed authentication across microservices and mobile apps.

Decision factors

Pattern	Stateless	User Context	Immediate Revocation	Horizontal Scaling
API Key	Often no (DB lookup)	No	Easy if server-checked	Excellent (shared cache)
Session	No	Rich	Excellent	Requires shared storage
JWT	Yes (local verify)	Embedded	Complex	Excellent

Hybrid approaches

Production systems often combine methods. GitHub uses personal access tokens (user-generated API keys) for their REST API, opaque OAuth tokens for authorization, and sessions for their web interface. This optimizes each authentication method for its specific use case rather than forcing a single solution.